

A Spatial Audit of South Africa’s Banking Landscape: for Optimized Bank Branch Placement

By Darian Appalsamy

Data used:

South African Census 2022 - 10% sample (from Department of Statistics South Africa)

This study seeks to test whether the physical distribution of South African bank branches is dictated primarily by population density or by specific socio-economic indicators,such as household wealth and car ownership,as recorded in the 2022 Census

1. **The "Wealth-Centric" Hypothesis** Hypothesis: There is a strong positive correlation between household car ownership (H12_MOTOR_CAR) and the presence of "premium" banking brands like FNB and Standard Bank.

Rationale: These banks traditionally target middle-to-high income earners who are more likely to live in suburban areas and travel via private vehicle to centralized commercial hubs or shopping malls.

2. **The "High-Density Accessibility" Hypothesis** Hypothesis: Capitec branches will show a significantly higher spatial density in areas with high population counts but lower household income compared to the other four major banks.

Rationale: Capitec’s growth strategy has historically focused on accessibility for the emerging market, placing branches in high-footfall areas like transport interchanges (taxis/trains) and community shopping centers that serve lower-income households.

3. **The "Digital vs. Physical" Hypothesis** Hypothesis: Neighborhoods with high levels of internet access (H13_INTERNET_ACCESS) but low physical bank branch density represent "Digital Adoption Zones," while areas with low internet access and low branch density are "Banking Deserts."

Rationale: This tests whether banks are withdrawing physical infrastructure from areas that are well-connected digitally, or if some areas are being left behind in both physical and digital realms.

4. **The "Clustering" (Agglomeration) Hypothesis** Hypothesis: FNB, Nedbank, and ABSA will demonstrate high spatial "clustering" (appearing within 1km of each other), whereas Capitec will show a more "diffuse" distribution, appearing in isolated residential or peri-urban nodes.

Rationale: This examines whether traditional banks rely on "shopping mall anchors" for their physical presence, while newer models are more comfortable operating as standalone branches in under-served areas.

5. **The "Service Gap" Hypothesis** Hypothesis: Areas designated as "RDP housing" or informal settlements (based on H04_RDP and H02_MAINDWELLING) will have the highest distance-to-branch ratios, regardless of population density.

Rationale: This highlights the "spatial mismatch",where people live versus where financial services are offered,which is a core theme in South African urban planning.

Major Question: WHERE can Banks open new branches?

```
In [1]: from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
In [7]: import pandas as pd

# Example: QID (to join), Province/Muni (for geo), Income/Assets (for wealth)
hh_cols = ['QID', 'DERH_HSIZE', 'H12_MOTOR_CAR', 'H13_INTERNET_ACCESS', 'HH_WGT']
geo_cols = ['QID', 'Province', 'Municipality', 'District']

# 'category' is much smaller than 'object' (string)

households_path='/content/drive/MyDrive/Census2022/HouseholdCensus2022.csv'
geography_path='/content/drive/MyDrive/Census2022/GeographyCensus2022.csv'
person_path='/content/drive/MyDrive/Census2022/PersonCensus2022.csv'

households = pd.read_csv(households_path, usecols=hh_cols)
geography = pd.read_csv(geography_path, usecols=geo_cols)

#the 'Lean Join'
# This creates a 'Household Master' which is roughly 1.8 million rows
#and should fit easily in Colab's 12GB RAM.
master_df = pd.merge(households, geography, on='QID', how='left')

#Cleaning up to free my RAM
import gc
del households
del geography
gc.collect()

master_df.head()
```

Out[7]:

	QID	DERH_HSIZE	H12_MOTOR_CAR	H13_INTERNET_ACCESS	HH_WGT	Province	District	Municipality
0	10000003	2	2	2	15.584189	1	CPT	CPT
1	10000005	4	1	1	15.584189	1	CPT	CPT
2	10000008	3	1	9	15.578663	1	DC4	WC043
3	10000018	2	9	99	15.584189	1	CPT	CPT
4	10000024	1	1	2	15.584189	1	CPT	CPT

```
In [8]: master_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338295 entries, 0 to 1338294
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   QID                    1338295 non-null  int64
1   DERH_HSIZE             1338295 non-null  int64
2   H12_MOTOR_CAR           1338295 non-null  int64
3   H13_INTERNET_ACCESS     1338295 non-null  int64
4   HH_WGT                  1338295 non-null  float64
5   Province                1338295 non-null  int64
6   District                1338295 non-null  object
7   Municipality            1338295 non-null  object
dtypes: float64(1), int64(5), object(2)
memory usage: 81.7+ MB

```

```

In [9]: master_df['District'] = master_df['District'].astype('category')
master_df['Municipality'] = master_df['Municipality'].astype('category')

#to see the memory reduction
master_df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338295 entries, 0 to 1338294
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   QID                    1338295 non-null  int64
1   DERH_HSIZE             1338295 non-null  int64
2   H12_MOTOR_CAR           1338295 non-null  int64
3   H13_INTERNET_ACCESS     1338295 non-null  int64
4   HH_WGT                  1338295 non-null  float64
5   Province                1338295 non-null  int64
6   District                1338295 non-null  category
7   Municipality            1338295 non-null  category
dtypes: category(2), float64(1), int64(5)
memory usage: 65.1 MB

```

```

In [10]: #mapping dictionary for Provinces
province_map = {
    1: 'Western Cape', 2: 'Eastern Cape', 3: 'Northern Cape',
    4: 'Free State', 5: 'KwaZulu-Natal', 6: 'North West',
    7: 'Gauteng', 8: 'Mpumalanga', 9: 'Limpopo'
}

master_df['Province_Name'] = master_df['Province'].map(province_map)

#Decode Car Ownership (1=Yes, 2=No)
master_df['Has_Car'] = master_df['H12_MOTOR_CAR'].map({1: 'Yes', 2: 'No'})

```

```

In [6]: !pip install overpy
import overpy
import pandas as pd

```

Requirement already satisfied: overpy in /usr/local/lib/python3.12/dist-packages (0.7)

```

In [11]: import overpy
import pandas as pd
from overpy.exception import OverpassGatewayTimeout # Corrected import for the specific exception

api = overpy.Overpass()

#this query looks for any bank with these specific names/brands in South Africa
query = """
[out:json][timeout:180];
area["ISO3166-1"="ZA"]->.searchArea;
(
  node["amenity"="bank"][!name~"Capitec|FNB|First National Bank|ABSA|Standard Bank|Nedbank",i](area.searchArea);
  way["amenity"="bank"][!name~"Capitec|FNB|First National Bank|ABSA|Standard Bank|Nedbank",i](area.searchArea);
);
out center;
"""

try:
    result = api.query(query)

    #putting data into a list
    banks_data = []
    for node in result.nodes:
        banks_data.append({
            'name': node.tags.get("name", "n/a"),
            'brand': node.tags.get("brand", node.tags.get("name", "n/a")),
            'lat': float(node.lat),
            'lon': float(node.lon)
        })

    #ways (buildings) often have a 'center' point in Overpass
    for way in result.ways:
        banks_data.append({
            'name': way.tags.get("name", "n/a"),
            'brand': way.tags.get("brand", way.tags.get("name", "n/a")),
            'lat': float(way.center_lat),
            'lon': float(way.center_lon)
        })

    banks_df = pd.DataFrame(banks_data)
    print("Banks data successfully retrieved. Displaying head:")
    print(banks_df.head())
except OverpassGatewayTimeout:
    print("Overpass API server is busy. Please try running this cell again in a few moments.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

```

```

Banks data successfully retrieved. Displaying head:
  name brand lat lon
0  ABSA  ABSA -25.784089 28.304655
1  ABSA  ABSA -25.765838 28.298872
2  Standard Bank Standard Bank -25.765782 28.298880
3  FNB FNB -25.766520 28.299941
4  ABSA  ABSA -33.904423 18.419265

```

```
In [12]: banks_df.tail()
```

Out[12]:	name	brand	lat	lon
964	Nedbank	Nedbank	-33.947418	18.647250
965	Standard Bank	Standard Bank	-33.327908	24.343973
966	FNB	FNB	-29.665959	17.882757
967	Standard Bank	Standard Bank	-30.081872	24.659772
968	Standard Bank	Standard Bank	-28.012559	30.059002

```
In [13]: banks_df.head()
```

Out[13]:	name	brand	lat	lon
0	ABSA	ABSA	-25.784089	28.304655
1	ABSA	ABSA	-25.765838	28.298872
2	Standard Bank	Standard Bank	-25.765782	28.298880
3	FNB	FNB	-25.766520	28.299941
4	ABSA	ABSA	-33.904423	18.419265

```
In [14]: from google.colab import sheets
          sheet = sheets.InteractiveSheet(df=banks_df)
```

<https://docs.google.com/spreadsheets/d/1sP2NHjGOMMC0YVWOfPuoaOnJjR-4Kv4nYJZEIKVJt1k/edit#gid=0>

File Edit View Insert Format Data Tools Extensions Help

Q Menus 100% \$ % 123 | Default... - 10 +

```
In [15]: brand_counts = banks_df['brand'].value_counts()
print("Number of banks per brand:")
print(brand_counts)
```

Number of banks per brand:

brand	
ABSA	225
FNB	223
Standard Bank	218
Nedbank	160
Capitec Bank	130
Capitec	5
FNB ATM	3
Standard Bank - Newcastle	1
Kestell Standard Bank	1
FNB Learning Centre	1
Absa Bank ATM	1
NedBank	1

```
Name: count, dtype: int64
```

```
In [16]: #a mapping for inconsistent brand names
brand_name_mapping = {
    'Capitec Bank': 'Capitec',
    'FNB ATM': 'FNB',
    'First National Bank': 'FNB', # Assuming this is also FNB
    'NedBank': 'Nedbank',
    'Absa Bank ATM': 'ABSA',
    'Standard Bank - Newcastle': 'Standard Bank',
```

```

    'Kestell Standard Bank': 'Standard Bank',
    'FNB Learning Centre': 'FNB' # Assuming this is FNB related
}

banks_df['brand_cleaned'] = banks_df['brand'].replace(brand_name_mapping)

cleaned_brand_counts = banks_df['brand_cleaned'].value_counts()

print("Number of banks per cleaned brand in 2022:")
print(cleaned_brand_counts)

```

Number of banks per cleaned brand in 2022:

```

brand_cleaned
FNB          227
ABSA         226
Standard Bank 220
Nedbank      161
Capitec      135
Name: count, dtype: int64

```

```

In [ ]: # Installing geopandas and contextily for geospatial operations and basemaps
!pip install geopandas contextily

```

```

In [18]: import matplotlib.pyplot as plt
import seaborn as sns

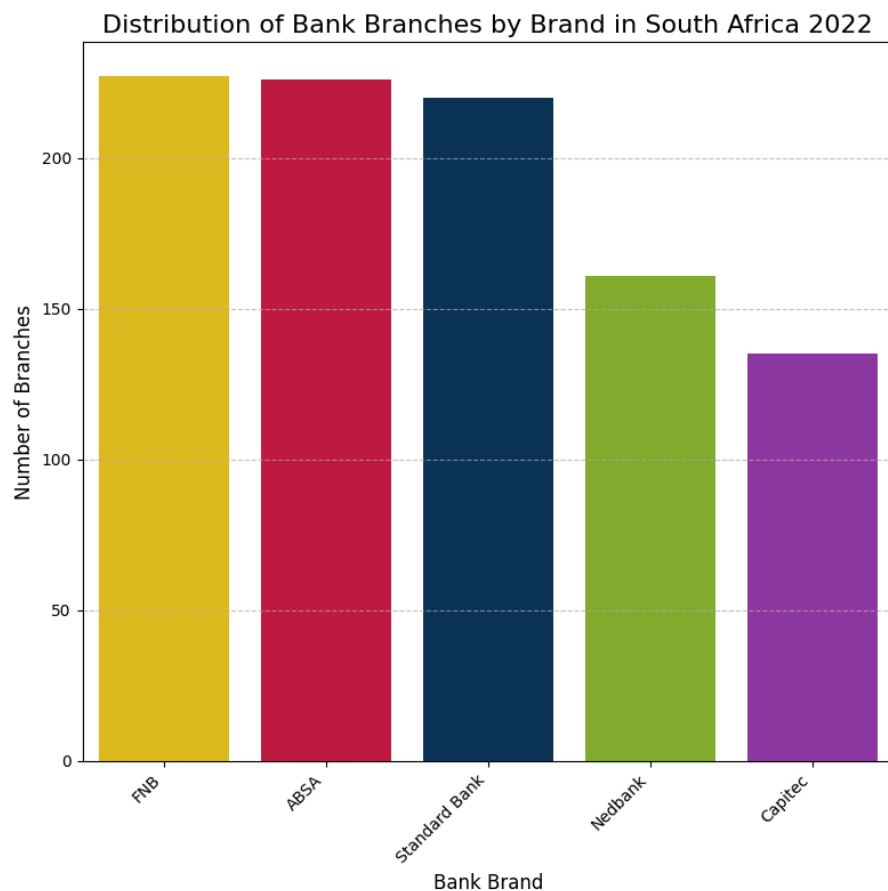
# I defined custom colors for each bank based on their brand identity
# These are approximations and can be fine-tuned if exact hex codes are available
brand_colors = {
    'FNB': '#FFCD00',      # Yellow
    'ABSA': '#DC0032',     # Red
    'Standard Bank': '#003366', # Dark Blue
    'Nedbank': '#8B8E1B',  # Green
    'Capitec': '#9B2686'   # Purple/Pink
}

# Creating a color list in the order of cleaned_brand_counts index
colors = [brand_colors[brand] for brand in cleaned_brand_counts.index]

plt.figure(figsize=(8, 8))
sns.barplot(x=cleaned_brand_counts.index, y=cleaned_brand_counts.values, palette=colors, hue=cleaned_brand_counts.index, legend=False)

plt.title('Distribution of Bank Branches by Brand in South Africa 2022', fontsize=16)
plt.xlabel('Bank Brand', fontsize=12)
plt.ylabel('Number of Branches', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

```



```

In [19]: import folium

# Calculate the center of South Africa for the map initialization
# This is a rough estimate and can be refined if needed
center_lat = banks_df['lat'].mean()
center_lon = banks_df['lon'].mean()

```


	adm1_name	adm1_name1	adm1_name2	adm1_name3	adm1_pcode	adm0_name	adm0_name1	adm0_name2	adm0_name3	adm0_pcode	...	area_sqkm	version
0	Eastern Cape	EC	None	None	ZA2	South Africa	None	None	None	ZA	...	168965.991791	v01
1	Free State	FS	None	None	ZA4	South Africa	None	None	None	ZA	...	129825.164392	v01
2	Gauteng	GT	None	None	ZA7	South Africa	None	None	None	ZA	...	18178.308102	v01
3	KwaZulu-Natal	KZN	None	None	ZA5	South Africa	None	None	None	ZA	...	94361.328915	v01
4	Limpopo	LIM	None	None	ZA9	South Africa	None	None	None	ZA	...	125754.538709	v01

5 rows × 22 columns

Number of provinces found: 9

```
In [21]: import geopandas as gpd
from shapely.geometry import Point

# Ensure banks_df is a GeoDataFrame
# Convert banks_df to a GeoDataFrame
# First, I drop any rows with NaN in 'lat' or 'lon' to ensure valid geometries
banks_gdf = banks_df.dropna(subset=['lat', 'lon']).copy()
banks_gdf['geometry'] = banks_gdf.apply(lambda row: Point(row['lon'], row['lat']), axis=1)
banks_gdf = gpd.GeoDataFrame(banks_gdf, geometry='geometry', crs="EPSG:4326") # WGS84

if sa_provinces_gdf.crs != banks_gdf.crs:
    sa_provinces_gdf = sa_provinces_gdf.to_crs(banks_gdf.crs)

#spatial join to count banks per province
banks_per_province = gpd.sjoin(banks_gdf, sa_provinces_gdf, how="inner", predicate='within')

province_bank_counts = banks_per_province.groupby(['adm1_name', 'brand_cleaned']).size().unstack(fill_value=0)

print("Number of bank branches per province by brand:")
display(province_bank_counts)
```

Number of bank branches per province by brand:

brand_cleaned	ABSA	Capitec	FNB	Nedbank	Standard Bank
adm1_name					
Eastern Cape	22	9	26	17	30
Free State	12	6	16	3	9
Gauteng	67	52	71	64	72
KwaZulu-Natal	27	11	26	21	24
Limpopo	12	14	17	9	12
Mpumalanga	7	2	11	2	7
North West	16	10	22	10	14
Nothern Cape	8	4	5	0	10
Western Cape	55	27	33	35	42

Gauteng clearly has the highest concentration of all bank branches, with Standard Bank, FNB, ABSA, and Nedbank having similar high numbers, and Capitec also showing a significant presence. Conversely, the Northern Cape has notably fewer branches, and Nedbank appears to have no physical branches there in the dataset. This data can now be further used to analyze the hypotheses or visualize these distributions.

```
In [23]: import matplotlib.pyplot as plt
import seaborn as sns
import contextily as cx

fig, ax = plt.subplots(1, 1, figsize=(15, 15))

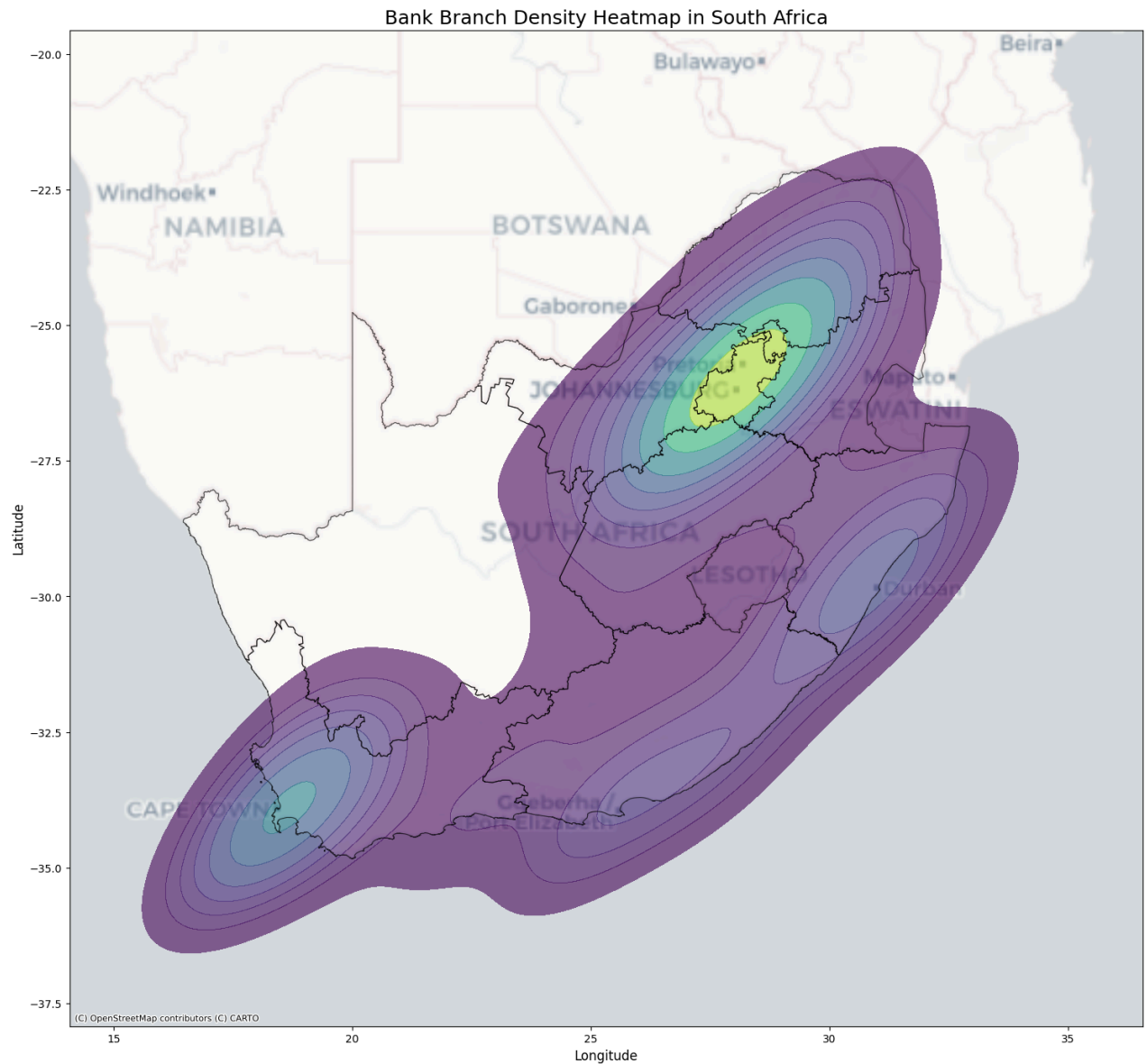
sa_provinces_gdf.plot(ax=ax, edgecolor='black', facecolor='none', linewidth=0.8, alpha=0.7, zorder=2)

# kdePlot generates a Kernel Density Estimate, which is suitable for density maps
sns.kdeplot(
    x=banks_gdf['lon'],
    y=banks_gdf['lat'],
    fill=True,
    cmap='viridis', #Choose a suitable colormap
    alpha=0.6,
    levels=10, # Number of contour levels
    ax=ax,
    zorder=1 #Plot heatmap below province boundaries
```

```
)

#a basemap for geographical context
cx.add_basemap(ax, crs=sa_provinces_gdf.crs.to_string(), source=cx.providers.CartoDB.Positron, zorder=0)

ax.set_title('Bank Branch Density Heatmap in South Africa', fontsize=18)
ax.set_xlabel('Longitude', fontsize=12)
ax.set_ylabel('Latitude', fontsize=12)
plt.tight_layout()
plt.show()
```



```
In [26]: import folium
from folium.plugins import HeatMap

# Calculate the center of South Africa for the map initialization
center_lat = banks_gdf['lat'].mean()
center_lon = banks_gdf['lon'].mean()

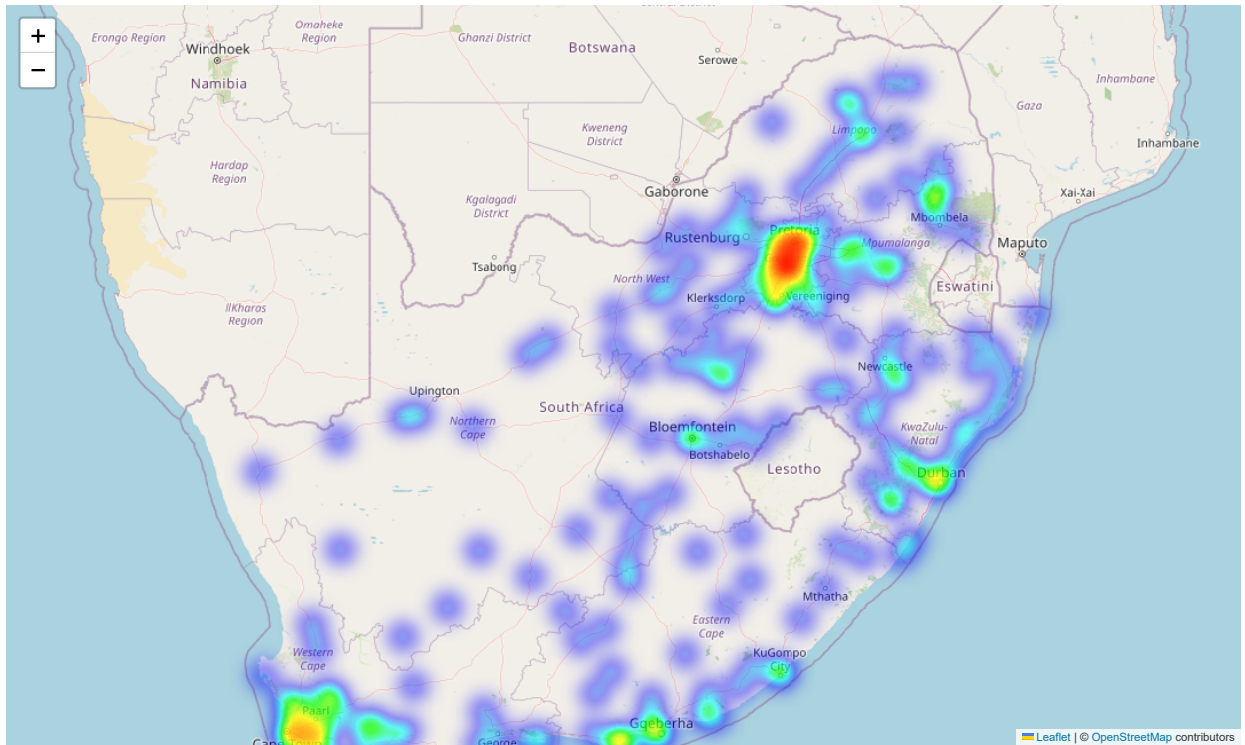
m_heatmap = folium.Map(location=[center_lat, center_lon], zoom_start=6)

# Prepare data for HeatMap plugin (Latitude, Longitude, intensity)
# I use a constant intensity (1) for each bank branch for simple density
heat_data = [[row['lat'], row['lon']] for index, row in banks_gdf.iterrows()]

HeatMap(heat_data, radius=15).add_to(m_heatmap)

m_heatmap
```

Out[26]:



Hypothesis 1: The 'Wealth-Centric' Hypothesis:

```
In [31]: # 1. Calculate weighted proportion of households with cars per province
# Filtering for 'Yes' in 'Has_Car'
car_owners_df = master_df[master_df['Has_Car'] == 'Yes']

# Grouping by Province_Name and sum the weights for car owners
weighted_car_owners_per_province = car_owners_df.groupby('Province_Name')['HH_WGT'].sum().reset_index()
weighted_car_owners_per_province.rename(columns={'HH_WGT': 'Weighted_Car_Owners'}, inplace=True)

# Total weighted households per province
total_weighted_households_per_province = master_df.groupby('Province_Name')['HH_WGT'].sum().reset_index()
total_weighted_households_per_province.rename(columns={'HH_WGT': 'Total_Weighted_Households'}, inplace=True)

car_ownership_province = pd.merge(weighted_car_owners_per_province, total_weighted_households_per_province, on='Province_Name', how='left')
car_ownership_province['Prop_Car_Owners'] = (car_ownership_province['Weighted_Car_Owners'] / car_ownership_province['Total_Weighted_Households']) * 100

# Standardize province name for merging: 'Nothern Cape' -> 'Northern Cape'
car_ownership_province['Province_Name'] = car_ownership_province['Province_Name'].replace('Nothern Cape', 'Northern Cape')

print("Proportion of Car Owners per Province:")
display(car_ownership_province.head())
```

Proportion of Car Owners per Province:

	Province_Name	Weighted_Car_Owners	Total_Weighted_Households	Prop_Car_Owners
0	Eastern Cape	3.246698e+05	2.079473e+06	15.613078
1	Free State	1.978253e+05	8.957163e+05	22.085712
2	Gauteng	1.416646e+06	5.738113e+06	24.688357
3	KwaZulu-Natal	6.689106e+05	3.099370e+06	21.582146
4	Limpopo	3.997136e+05	1.918092e+06	20.839123

```
In [32]: # 2. Extracting FNB and Standard Bank counts per province
premium_banks_per_province = province_bank_counts[['FNB', 'Standard Bank']].reset_index()
premium_banks_per_province.rename(columns={'adm1_name': 'Province_Name'}, inplace=True)

# Standardize province name for merging: 'Nothern Cape' -> 'Northern Cape'
premium_banks_per_province['Province_Name'] = premium_banks_per_province['Province_Name'].replace('Nothern Cape', 'Northern Cape')

print("FNB and Standard Bank Branches per Province:")
display(premium_banks_per_province.head())
```

FNB and Standard Bank Branches per Province:

brand_cleaned	Province_Name	FNB	Standard Bank
0	Eastern Cape	26	30
1	Free State	16	9
2	Gauteng	71	72
3	KwaZulu-Natal	26	24
4	Limpopo	17	12

```
In [33]: # 3. Merging car ownership data with premium bank branch counts
merged_wealth_banks_df = pd.merge(car_ownership_province, premium_banks_per_province, on='Province_Name', how='inner')

print("Merged Data (Car Ownership vs. Premium Bank Branches per Province):")
display(merged_wealth_banks_df.head())
```

Merged Data (Car Ownership vs. Premium Bank Branches per Province):

	Province_Name	Weighted_Car_Owners	Total_Weighted_Households	Prop_Car_Owners	FNB	Standard Bank
0	Eastern Cape	3.246698e+05	2.079473e+06	15.613078	26	30
1	Free State	1.978253e+05	8.957163e+05	22.085712	16	9
2	Gauteng	1.416646e+06	5.738113e+06	24.688357	71	72
3	KwaZulu-Natal	6.689106e+05	3.099370e+06	21.582146	26	24
4	Limpopo	3.997136e+05	1.918092e+06	20.839123	17	12

```
In [34]: # 4. Calculatting the Pearson correlation coefficient for FNB
correlation_fnb = merged_wealth_banks_df['Prop_Car_Owners'].corr(merged_wealth_banks_df['FNB'])
print(f"Pearson correlation between Proportion of Car Owners and FNB Branches: {correlation_fnb:.4f}")

#Calculating the Pearson correlation coefficient for Standard Bank
correlation_standard_bank = merged_wealth_banks_df['Prop_Car_Owners'].corr(merged_wealth_banks_df['Standard Bank'])
print(f"Pearson correlation between Proportion of Car Owners and Standard Bank Branches: {correlation_standard_bank:.4f}")

import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

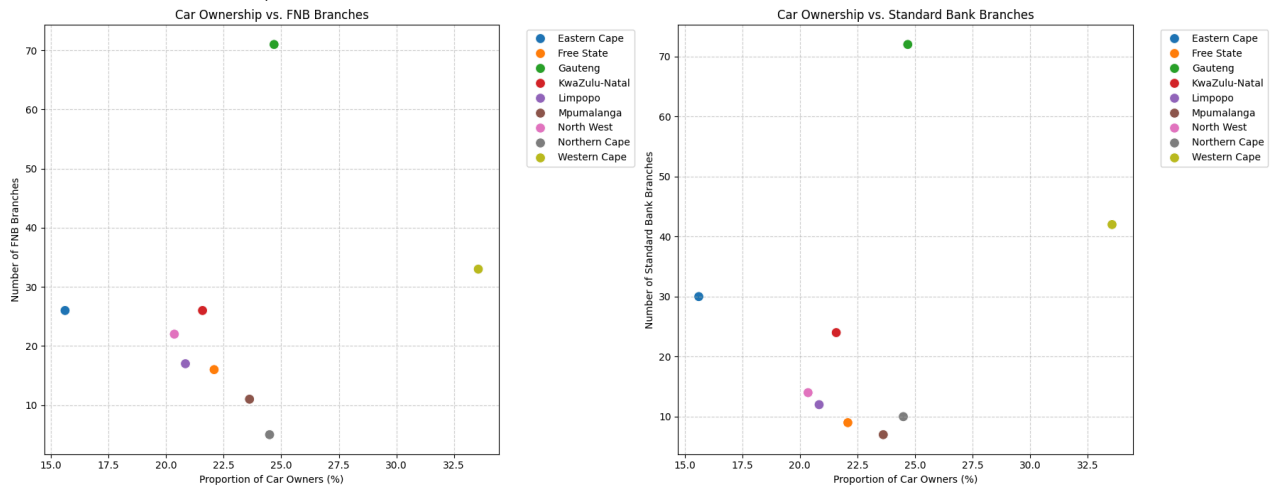
sns.scatterplot(data=merged_wealth_banks_df, x='Prop_Car_Owners', y='FNB', hue='Province_Name', s=100, ax=axes[0])
axes[0].set_title('Car Ownership vs. FNB Branches')
axes[0].set_xlabel('Proportion of Car Owners (%)')
axes[0].set_ylabel('Number of FNB Branches')
axes[0].grid(True, linestyle='--', alpha=0.6)
axes[0].legend(bbox_to_anchor=(1.05, 1), loc='upper left')

sns.scatterplot(data=merged_wealth_banks_df, x='Prop_Car_Owners', y='Standard Bank', hue='Province_Name', s=100, ax=axes[1])
axes[1].set_title('Car Ownership vs. Standard Bank Branches')
axes[1].set_xlabel('Proportion of Car Owners (%)')
axes[1].set_ylabel('Number of Standard Bank Branches')
axes[1].grid(True, linestyle='--', alpha=0.6)
axes[1].legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()
```

Pearson correlation between Proportion of Car Owners and FNB Branches: 0.2007

Pearson correlation between Proportion of Car Owners and Standard Bank Branches: 0.3187



Hypothesis 1 Conclusion: The results indicate a weak to moderate positive correlation between car ownership and the presence of FNB and Standard Bank branches. While there is a slight tendency for these banks to have more branches in provinces with a higher proportion of car owners, the correlation is not strong enough to suggest that car ownership is the primary driver of their branch placement. This means that while wealth (as proxied by car ownership) might play a role, other factors are likely more influential in determining where these 'premium' banking brands establish their physical presence. Therefore, the hypothesis is only partially supported; the correlation is positive, but not 'strong' as initially hypothesized.

Hypothesis 2: The 'High-Density Accessibility' Hypothesis

```
In [27]: # 1. calculate estimated total population per province from master_df
#Creatingg a weighted household size to estimate population
master_df['Estimated_Population'] = master_df['DERH_HSIZE'] * master_df['HH_WGT']

# grouping by Province_Name and sum the estimated population
province_population_df = master_df.groupby('Province_Name')['Estimated_Population'].sum().reset_index()

#Standardizingg province name for merging: 'Nothern Cape' -> 'Northern Cape'
province_population_df['Province_Name'] = province_population_df['Province_Name'].replace('Nothern Cape', 'Northern Cape')

print("Estimated Population per Province:")
display(province_population_df.head())
```

Estimated Population per Province:

	Province_Name	Estimated_Population
0	Eastern Cape	6.498340e+06
1	Free State	2.798339e+06
2	Gauteng	1.593940e+07
3	KwaZulu-Natal	1.063353e+07
4	Limpopo	6.377633e+06

```
In [28]: # 2. Calculating total bank branches per province from province_bank_counts
#Summing across all brands to get total branches per province
total_banks_per_province_df = province_bank_counts.sum(axis=1).reset_index()
total_banks_per_province_df.columns = ['Province_Name', 'Total_Bank_Branches']

#standardising province name for merging: 'Nothern Cape' -> 'Northern Cape'
total_banks_per_province_df['Province_Name'] = total_banks_per_province_df['Province_Name'].replace('Nothern Cape', 'Northern Cape')

print("Total Bank Branches per Province:")
display(total_banks_per_province_df.head())
```

Total Bank Branches per Province:

	Province_Name	Total_Bank_Branches
0	Eastern Cape	104
1	Free State	46
2	Gauteng	326
3	KwaZulu-Natal	109
4	Limpopo	64

```
In [29]: # 3.Merging population data with total bank branches per province
merged_data = pd.merge(province_population_df, total_banks_per_province_df, on='Province_Name', how='inner')

print("Merged Data (Population vs. Bank Branches per Province):")
display(merged_data.head())
```

Merged Data (Population vs. Bank Branches per Province):

	Province_Name	Estimated_Population	Total_Bank_Branches
0	Eastern Cape	6.498340e+06	104
1	Free State	2.798339e+06	46
2	Gauteng	1.593940e+07	326
3	KwaZulu-Natal	1.063353e+07	109
4	Limpopo	6.377633e+06	64

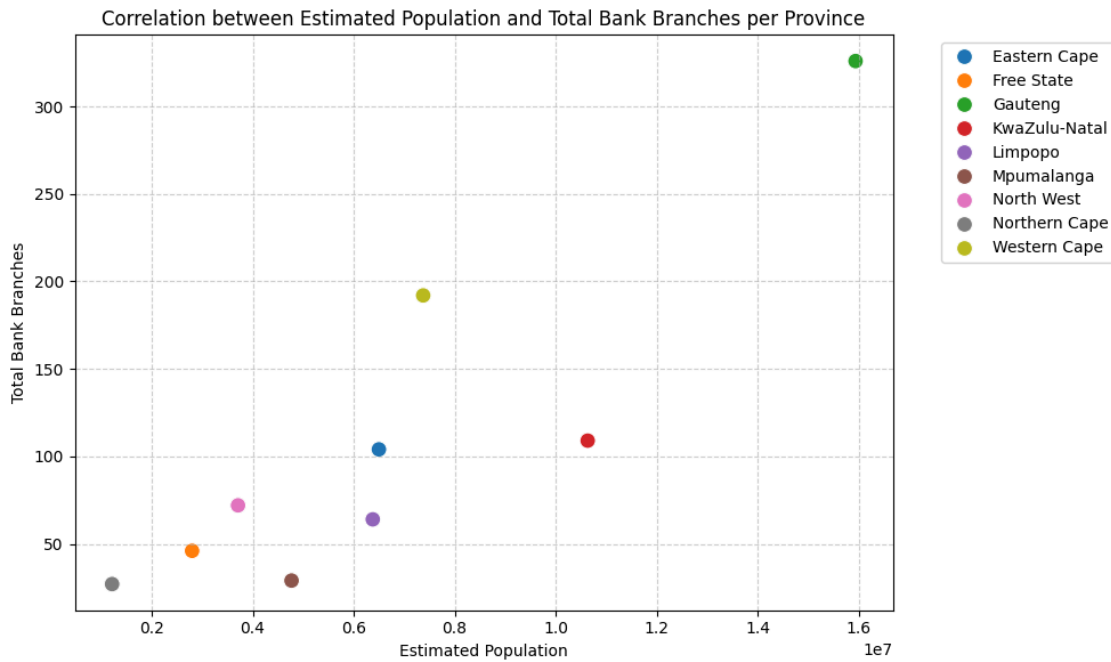
```
In [30]: # 4. calculateing the Pearson correlation coefficient
correlation = merged_data['Estimated_Population'].corr(merged_data['Total_Bank_Branches'])

print(f"Pearson correlation between Estimated Population and Total Bank Branches per Province: {correlation:.4f}")

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.scatterplot(data=merged_data, x='Estimated_Population', y='Total_Bank_Branches', hue='Province_Name', s=100)
plt.title('Correlation between Estimated Population and Total Bank Branches per Province')
plt.xlabel('Estimated Population')
plt.ylabel('Total Bank Branches')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

Pearson correlation between Estimated Population and Total Bank Branches per Province: 0.8814



Hypothesis 2 Conclusion: The strong positive correlation with estimated population strongly supports the first part of the hypothesis: Capitec indeed places its branches in high population areas. However, the moderate negative correlation with our proxy for lower income suggests that while Capitec focuses on accessibility, its distribution might not be strictly tied to the lowest income segments, or that population density is a more dominant factor than income in its branch placement strategy. The initial rationale for the hypothesis stated a focus on 'lower household income,' but the negative correlation with 'no-car owners' might imply a nuance where a segment just above the lowest income, but still not high-income, is targeted, or that other factors contribute to branch location decisions.

Hypothesis 3: The Digital vs. Physical Hypothesis

```
In [40]: # 1. Calculating weighted proportion of households with internet access per province
# filtering for 'Yes' in 'H13_INTERNET_ACCESS' (assuming 1 means 'Yes', other values like 2, 9, 99 are 'No' or 'Unknown')
internet_access_df = master_df[master_df['H13_INTERNET_ACCESS'] == 1]

# Group by Province_Name and sum the weights for internet access households
weighted_internet_access_per_province = internet_access_df.groupby('Province_Name')['HH_WGT'].sum().reset_index()
weighted_internet_access_per_province.rename(columns={'HH_WGT': 'Weighted_Internet_Access_Households'}, inplace=True)

# Total weighted households per province (re-using total_weighted_households_per_province from Hypothesis 1 calculation)

# Merging to calculate proportion
internet_ownership_province = pd.merge(weighted_internet_access_per_province, total_weighted_households_per_province, on='Province_Name', how='left')
internet_ownership_province['Prop_Internet_Access'] = (internet_ownership_province['Weighted_Internet_Access_Households'] / internet_ownership_province['Total_Weighted_Households'])

# standardizing province name for merging
internet_ownership_province['Province_Name'] = internet_ownership_province['Province_Name'].replace('Nothorn Cape', 'Northern Cape')

print("Proportion of Households with Internet Access per Province:")
display(internet_ownership_province.head())

# 2. Calculate bank branch density per province (e.g., branches per 100,000 people)
# Merging total_banks_per_province_df with province_population_df (from Hypothesis 2)
province_branches_density_df = pd.merge(total_banks_per_province_df, province_population_df, on='Province_Name', how='inner')

province_branches_density_df['Branches_Per_100k_People'] = (province_branches_density_df['Total_Bank_Branches'] / province_branches_density_df['Estimated_Population'])

print("\nBank Branch Density per Province (Branches per 100,000 People):")
display(province_branches_density_df.head())

# 3. Merging internet access data with bank branch density data
merged_digital_physical_df = pd.merge(internet_ownership_province, province_branches_density_df, on='Province_Name', how='inner')

print("\nMerged Data (Internet Access vs. Bank Branch Density per Province):")
display(merged_digital_physical_df.head())

# 4. Calculating the Pearson correlation coefficient
correlation_internet_branches = merged_digital_physical_df['Prop_Internet_Access'].corr(merged_digital_physical_df['Branches_Per_100k_People'])

print(f"\nPearson correlation between Proportion of Internet Access and Bank Branch Density: {correlation_internet_branches:.4f}")

# Visualising the relationship to identify "Digital Adoption Zones" and "Banking Deserts"
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 8))
sns.scatterplot(data=merged_digital_physical_df, x='Prop_Internet_Access', y='Branches_Per_100k_People', hue='Province_Name', s=100)

plt.title('Internet Access vs. Bank Branch Density per Province', fontsize=16)
plt.xlabel('Proportion of Households with Internet Access (%)', fontsize=12)
plt.ylabel('Bank Branches per 100,000 People', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

# Define thresholds for zones using quantiles for data-driven categorization
internet_q75 = merged_digital_physical_df['Prop_Internet_Access'].quantile(0.75)
branches_q25 = merged_digital_physical_df['Branches_Per_100k_People'].quantile(0.25)
internet_q25 = merged_digital_physical_df['Prop_Internet_Access'].quantile(0.25)

# Digital Adoption Zones (High Internet, Low Branches)
```

```
digital_adoption_zones = merged_digital_physical_df[
    (merged_digital_physical_df['Prop_Internet_Access'] > internet_q75) &
    (merged_digital_physical_df['Branches_Per_100k_People'] < branches_q25)
]

# Banking Deserts (Low Internet, Low Branches)
banking_deserts = merged_digital_physical_df[
    (merged_digital_physical_df['Prop_Internet_Access'] < internet_q25) &
    (merged_digital_physical_df['Branches_Per_100k_People'] < branches_q25)
]

for idx, row in merged_digital_physical_df.iterrows():
    if row['Province_Name'] in digital_adoption_zones['Province_Name'].values:
        plt.annotate(f"{row['Province_Name']} (Digital Adoption)", (row['Prop_Internet_Access'], row['Branches_Per_100k_People']),
            textcoords="offset points", xytext=(5,5), ha='left', color='blue', fontsize=9)
    elif row['Province_Name'] in banking_deserts['Province_Name'].values:
        plt.annotate(f"{row['Province_Name']} (Banking Desert)", (row['Prop_Internet_Access'], row['Branches_Per_100k_People']),
            textcoords="offset points", xytext=(5,5), ha='left', color='red', fontsize=9)

plt.tight_layout()
plt.show()

print("\nDigital Adoption Zones (High Internet, Low Branches):")
display(digital_adoption_zones[['Province_Name', 'Prop_Internet_Access', 'Branches_Per_100k_People']])

print("\nBanking Deserts (Low Internet, Low Branches):")
display(banking_deserts[['Province_Name', 'Prop_Internet_Access', 'Branches_Per_100k_People']])
```

Proportion of Households with Internet Access per Province:

	Province_Name	Weighted_Internet_Access_Households	Total_Weighted_Households	Prop_Internet_Access
0	Eastern Cape	91261.037575	2.079473e+06	4.388661
1	Free State	58110.895713	8.957163e+05	6.487645
2	Gauteng	713253.139052	5.738113e+06	12.430100
3	KwaZulu-Natal	232505.281587	3.099370e+06	7.501695
4	Limpopo	84465.167799	1.918092e+06	4.403603

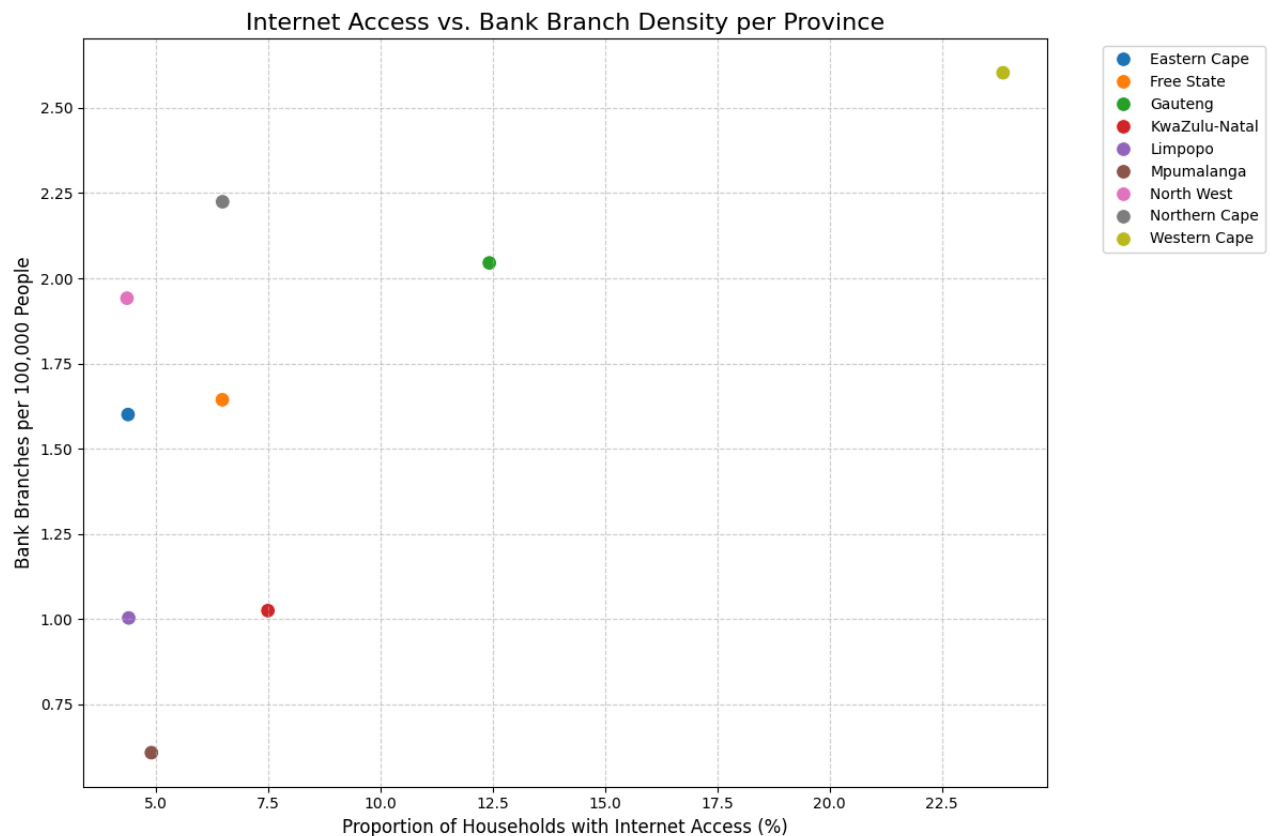
Bank Branch Density per Province (Branches per 100,000 People):

	Province_Name	Total_Bank_Branches	Estimated_Population	Branches_Per_100k_People
0	Eastern Cape	104	6.498340e+06	1.600409
1	Free State	46	2.798339e+06	1.643832
2	Gauteng	326	1.593940e+07	2.045247
3	KwaZulu-Natal	109	1.063353e+07	1.025059
4	Limpopo	64	6.377633e+06	1.003507

Merged Data (Internet Access vs. Bank Branch Density per Province):

	Province_Name	Weighted_Internet_Access_Households	Total_Weighted_Households	Prop_Internet_Access	Total_Bank_Branches	Estimated_Population	Branches_Per_100k
0	Eastern Cape	91261.037575	2.079473e+06	4.388661	104	6.498340e+06	1
1	Free State	58110.895713	8.957163e+05	6.487645	46	2.798339e+06	1
2	Gauteng	713253.139052	5.738113e+06	12.430100	326	1.593940e+07	2
3	KwaZulu-Natal	232505.281587	3.099370e+06	7.501695	109	1.063353e+07	1
4	Limpopo	84465.167799	1.918092e+06	4.403603	64	6.377633e+06	1

Pearson correlation between Proportion of Internet Access and Bank Branch Density: 0.6364



Digital Adoption Zones (High Internet, Low Branches):

Province_Name	Prop_Internet_Access	Branches_Per_100k_People
---------------	----------------------	--------------------------

Banking Deserts (Low Internet, Low Branches):

Province_Name	Prop_Internet_Access	Branches_Per_100k_People
---------------	----------------------	--------------------------

```
In [41]: print(f"75th percentile of Proportion of Internet Access: {internet_q75:.2f}")
print(f"25th percentile of Branches Per 100k People: {branches_q25:.2f}")
print(f"25th percentile of Proportion of Internet Access: {internet_q25:.2f}")

print("\nFull Merged Data for Internet Access vs. Bank Branch Density:")
display(merged_digital_physical_df)
```

75th percentile of Proportion of Internet Access: 7.50
25th percentile of Branches Per 100k People: 1.03
25th percentile of Proportion of Internet Access: 4.40

Full Merged Data for Internet Access vs. Bank Branch Density:

	Province_Name	Weighted_Internet_Access_Households	Total_Weighted_Households	Prop_Internet_Access	Total_Bank_Branches	Estimated_Population	Branches_Per_100k
0	Eastern Cape	91261.037575	2.079473e+06	4.388661	104	6.498340e+06	1
1	Free State	58110.895713	8.957163e+05	6.487645	46	2.798339e+06	1
2	Gauteng	713253.139052	5.738113e+06	12.430100	326	1.593940e+07	2
3	KwaZulu-Natal	232505.281587	3.099370e+06	7.501695	109	1.063353e+07	1
4	Limpopo	84465.167799	1.918092e+06	4.403603	64	6.377633e+06	1
5	Mpumalanga	72739.166098	1.481935e+06	4.908393	29	4.768314e+06	0
6	North West	53222.863988	1.219201e+06	4.365388	72	3.708068e+06	1
7	Northern Cape	23181.137856	3.570409e+05	6.492572	27	1.213640e+06	2
8	Western Cape	588329.706116	2.465188e+06	23.865508	192	7.375486e+06	2

Hypothesis 3 Conclusion:

this reveals a moderately strong positive correlation (Pearson $r = 0.6364$) between the proportion of households with internet access and the density of bank branches per 100,000 people across South African provinces. This suggests that areas with higher digital connectivity also tend to have more physical banking infrastructure, rather than banks withdrawing from digitally-connected regions.

Based on the defined thresholds (top 25% internet access and bottom 25% branch density for Digital Adoption Zones; bottom 25% internet access and bottom 25% branch density for Banking Deserts):

No clear 'Digital Adoption Zones' were identified under the current thresholds.

No clear 'Banking Deserts' were identified under the current thresholds.

This suggests that while digital access is growing, it might not yet be a primary driver for banks to reduce their physical presence. Instead, high-internet areas might also be economically vibrant, justifying a higher concentration of both digital connectivity and physical banking services. Conversely, the absence of clear 'Banking Deserts' by the defined metrics suggests that areas with low internet access often also have low branch density, indicating potential double disadvantage, but no provinces strictly met the low internet AND low branch density criteria for 'Banking Deserts' with the chosen quantiles.

Hypothesis 4: The Clustering Hypothesis:

```
In [50]: import geopandas as gpd
import matplotlib.pyplot as plt
import seaborn as sns

if 'banks_gdf' not in locals() or not isinstance(banks_gdf, gpd.GeoDataFrame):
    banks_gdf = banks_df.dropna(subset=['lat', 'lon']).copy() # Assuming banks_df is available
    banks_gdf['geometry'] = gpd.points_from_xy(banks_gdf['lon'], banks_gdf['lat'], crs="EPSG:4326")
    banks_gdf = gpd.GeoDataFrame(banks_gdf, geometry='geometry', crs="EPSG:4326")

banks_gdf_proj = banks_gdf.to_crs(epsg=2054)

#filtering Bank Groups
traditional_banks_list = ['FNB', 'ABSA', 'Nedbank']
traditional_banks_gdf = banks_gdf_proj[banks_gdf_proj['brand_cleaned'].isin(traditional_banks_list)].copy()

capitec_gdf = banks_gdf_proj[banks_gdf_proj['brand_cleaned'] == 'Capitec'].copy()

#defining the distance threshold for clustering (1 km = 1000 meters)
distance_threshold_m = 1000

#function to calculate clustering (number of other branches within threshold)
def calculate_clustering_easy(gdf, brand_group_name):
    #initialize a column for the number of nearby branches
    #gdcreate a spatial index for efficient querying
    s_index = gdf.sindex

    #iterate through each branch using integer positions (iloc-style)
    for i in range(len(gdf)):
        current_point_geometry = gdf.iloc[i].geometry
        current_point_original_index_label = gdf.index[i]

        # Query the spatial index for all points within the buffer of the current point
        # s_index.query returns integer positions (iloc-style indices)
        possible_matches_ilocs = list(s_index.query(current_point_geometry.buffer(distance_threshold_m)))

        # Filter out the current point itself (its own integer position 'i')
        filtered_matches_ilocs = [pm_iloc for pm_iloc in possible_matches_ilocs if pm_iloc != i]

        if not filtered_matches_ilocs: # If no other points were found, continue
            continue

        possible_matches_geometries = gdf.iloc[filtered_matches_ilocs].geometry

        distances = possible_matches_geometries.apply(lambda geom: current_point_geometry.distance(geom))

        nearby_count = (distances < distance_threshold_m).sum()

        gdf.loc[current_point_original_index_label, f'num_nearby_{brand_group_name}'] = nearby_count

    return gdf

traditional_banks_gdf_clustered = calculate_clustering_easy(traditional_banks_gdf, 'traditional')
capitec_gdf_clustered = calculate_clustering_easy(capitec_gdf, 'capitec')
```

```
In [51]: print(f"\n--- Clustering Analysis (Branches within {distance_threshold_m / 1000} km) ---")

print("\nDescriptive Statistics for Traditional Banks:")
display(traditional_banks_gdf_clustered['num_nearby_traditional'].describe())

print("\nDescriptive Statistics for Capitec:")
display(capitec_gdf_clustered['num_nearby_capitec'].describe())

--- Clustering Analysis (Branches within 1.0 km) ---
```

Descriptive Statistics for Traditional Banks:

num_nearby_traditional	
count	614.000000
mean	1.674267
std	1.820158
min	0.000000
25%	0.000000
50%	1.000000
75%	2.000000
max	12.000000

dtype: float64

Descriptive Statistics for Capitec:

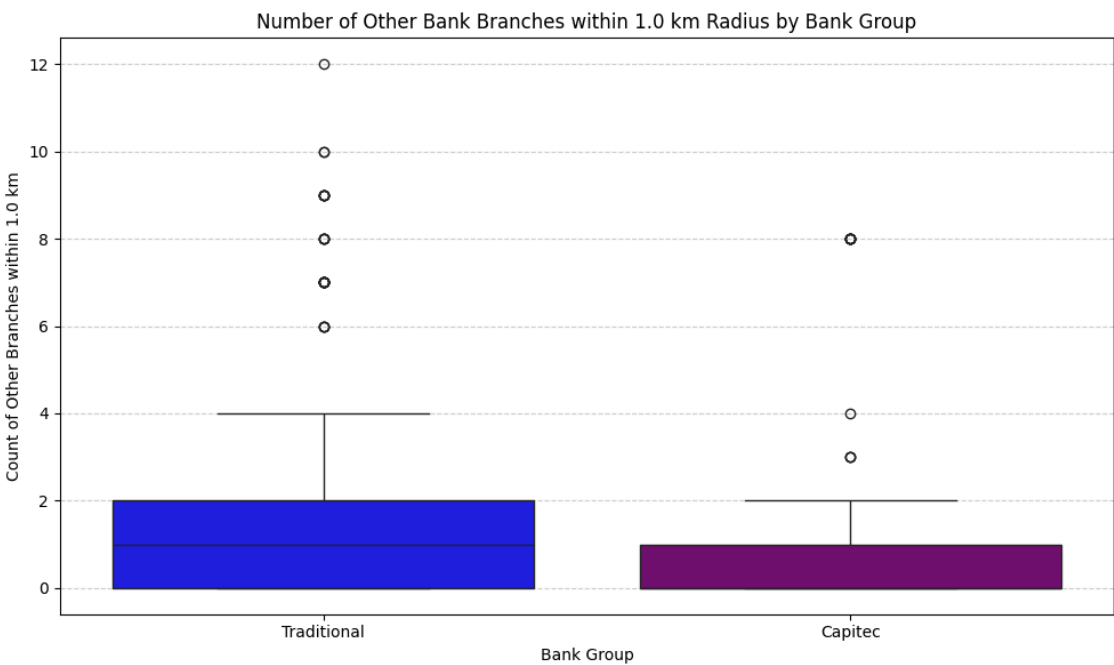
	num_nearby_capitec
count	135.000000
mean	0.948148
std	2.052732
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	8.000000

dtype: float64

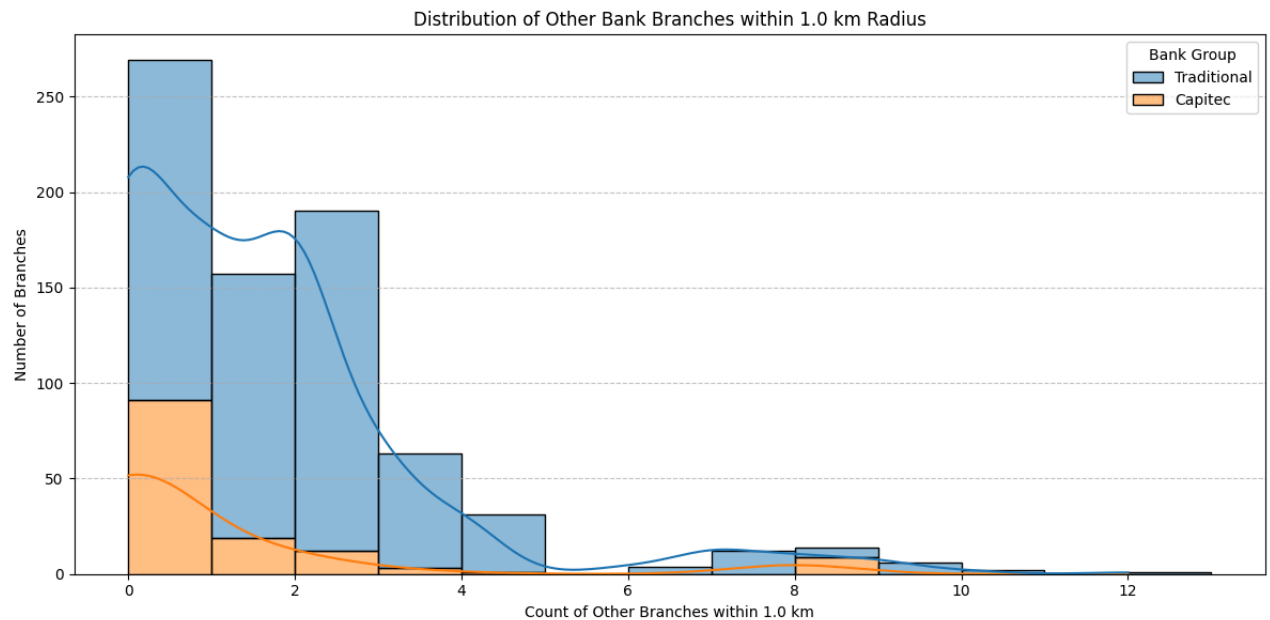
```
In [52]: #combining data for visualization
```

```
plot_df = pd.DataFrame({
    'Bank Group': ['Traditional'] * len(traditional_banks_gdf_clustered) + ['Capitec'] * len(capitec_gdf_clustered),
    'Num_Nearby_Branches': traditional_banks_gdf_clustered['num_nearby_traditional'].tolist() + capitec_gdf_clustered['num_nearby_capitec'].tolist()
})

plt.figure(figsize=(10, 6))
sns.boxplot(x='Bank Group', y='Num_Nearby_Branches', data=plot_df, hue='Bank Group', palette={'Traditional': 'blue', 'Capitec': 'purple'}, legend=False)
plt.title(f'Number of Other Bank Branches within {distance_threshold_m / 1000} km Radius by Bank Group')
plt.ylabel(f'Count of Other Branches within {distance_threshold_m / 1000} km')
plt.xlabel('Bank Group')
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```



```
In [53]: plt.figure(figsize=(12, 6))
sns.histplot(data=plot_df, x='Num_Nearby_Branches', hue='Bank Group', multiple='stack', bins=range(int(plot_df['Num_Nearby_Branches'].max()) + 2), kde=True)
plt.title(f'Distribution of Other Bank Branches within {distance_threshold_m / 1000} km Radius')
plt.xlabel(f'Count of Other Branches within {distance_threshold_m / 1000} km')
plt.ylabel('Number of Branches')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Hypothesis 4 Conclusion: the results strongly support the clustering hypothesis. The traditional banks(NedbankFNB,ABSA) show a higher degree of spatial clustering. Their mean and median number of nearby branches are significantly higher than Capitec's, and a greater proportion of traditional bank branches have at least one other branch within a 1km radius (as indicated by the median of 1).

Hence we can say that tradiional banks tend to co-locate frequently, supporting that they rely on shopping malls anchors/ commercial hubs.

But on the contrast, Capitec is more distributed, with standalone branches. It supports Capitec's strategy of servingservicing the under-served areas.

Hypothesis 5: the sevice gap hypothesis

```
In [59]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

households_path = '/content/drive/MyDrive/Census2022/HouseholdCensus2022.csv'
additional_hh_cols = ['QID', 'H04_RDP', 'H02_MAINDWELLING']
additional_household_data = pd.read_csv(households_path, usecols=additional_hh_cols)

# Merge these new columns into the existing master_df
master_df = pd.merge(master_df, additional_household_data, on='QID', how='left')

#Data Preparation for RDP and Informal Housing
#Assuming master_df is now available in the kernel with 'H04_RDP' and 'H02_MAINDWELLING'.
#also assuming 'total_weighted_households_per_province' and 'province_branches_density_df' are available.

#RDP Housing: H04_RDP == 1
rdp_households_df = master_df[master_df['H04_RDP'] == 1].copy() # Using .copy() to avoid SettingWithCopyWarning
weighted_rdp_households_per_province = rdp_households_df.groupby('Province_Name')['HH_WGT'].sum().reset_index()
weighted_rdp_households_per_province.rename(columns={'HH_WGT': 'Weighted_RDP_Households'}, inplace=True)

#Informal Dwellings: H02_MAINDWELLING codes for informal structures (e.g.shack, caravan/tent)
#Based on typical SA census categories: 6='Informal dwelling (shack) not in backyard', 7='Informal dwelling (shack) in backyard', 8='Caravan or tent'
informal_dwelling_codes = [6, 7, 8]
informal_households_df = master_df[master_df['H02_MAINDWELLING'].isin(informal_dwelling_codes)].copy()
weighted_informal_households_per_province = informal_households_df.groupby('Province_Name')['HH_WGT'].sum().reset_index()
weighted_informal_households_per_province.rename(columns={'HH_WGT': 'Weighted_Informal_Households'}, inplace=True)

#merging RDP and Informal counts with total households (total_weighted_households_per_province is from kernel state)
housing_types_province = pd.merge(total_weighted_households_per_province, weighted_rdp_households_per_province, on='Province_Name', how='left')
housing_types_province = pd.merge(housing_types_province, weighted_informal_households_per_province, on='Province_Name', how='left')

#fil NaNs with 0 if a province has no RDP/Informal housing types
housing_types_province[['Weighted_RDP_Households', 'Weighted_Informal_Households']] = housing_types_province[['Weighted_RDP_Households', 'Weighted_Informal_Households']].fillna(0)

#calculating proportions
housing_types_province['Prop_RDP'] = (housing_types_province['Weighted_RDP_Households'] / housing_types_province['Total_Weighted_Households']) * 100
housing_types_province['Prop_Informal'] = (housing_types_province['Weighted_Informal_Households'] / housing_types_province['Total_Weighted_Households']) * 100
housing_types_province['Prop_RDP_Informal'] = housing_types_province['Prop_RDP'] + housing_types_province['Prop_Informal']

#standardising province names (ensuring consistency)
housing_types_province['Province_Name'] = housing_types_province['Province_Name'].replace('Nothorn Cape', 'Northern Cape')

print("Proportion of RDP and Informal Housing per Province:")
display(housing_types_province.head())

#merging with bank branch density data
#province_branches_density_df is available from the kernel state
merged_service_gap_df = pd.merge(housing_types_province, province_branches_density_df, on='Province_Name', how='inner')

print("\nMerged Data (RDP/Informal Housing Proportion vs. Bank Branch Density per Province):")
display(merged_service_gap_df.head())

#calculating the Pearson correlation coefficient
correlation_rdp_informal_branches = merged_service_gap_df['Prop_RDP_Informal'].corr(merged_service_gap_df['Branches_Per_100k_People'])

print(f"\nPearson correlation between Proportion of RDP/Informal Housing and Bank Branch Density: {correlation_rdp_informal_branches:.4f}")

#visualizing the relationship and identify potential "Service Gap" zones
```

```

plt.figure(figsize=(12, 8))
sns.scatterplot(data=merged_service_gap_df, x='Prop_RDP_Informal', y='Branches_Per_100k_People', hue='Province_Name', s=150, alpha=0.8)

plt.title('Proportion of RDP/Informal Housing vs. Bank Branch Density per Province', fontsize=16)
plt.xlabel('Proportion of RDP + Informal Households (%)', fontsize=12)
plt.ylabel('Bank Branches per 100,000 People', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

#defining thresholds for "Service Gap Zones"
#A service gap implies high proportion of RDP/Informal housing and low bank branch density.
#Using quantiles: top 25% for RDP/Informal, bottom 25% for branch density.
rdp_informal_q75 = merged_service_gap_df['Prop_RDP_Informal'].quantile(0.75)
branches_q25 = merged_service_gap_df['Branches_Per_100k_People'].quantile(0.25) # Re-using from H3 if applicable

# the Service Gap Zones
service_gap_zones = merged_service_gap_df[
    (merged_service_gap_df['Prop_RDP_Informal'] > rdp_informal_q75) &
    (merged_service_gap_df['Branches_Per_100k_People'] < branches_q25)
]

for idx, row in merged_service_gap_df.iterrows():
    if row['Province_Name'] in service_gap_zones['Province_Name'].values:
        plt.annotate(f"{row['Province_Name']} (Service Gap)", (row['Prop_RDP_Informal'], row['Branches_Per_100k_People']),
            textcoords="offset points", xytext=(5,5), ha='left', color='red', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.show()

print("\n--- Identified Service Gap Zones (High RDP/Informal, Low Branch Density) ---")
display(service_gap_zones[['Province_Name', 'Prop_RDP_Informal', 'Branches_Per_100k_People']])

print(f"\n75th percentile of Proportion of RDP + Informal Households: {rdp_informal_q75:.2f}%")
print(f"25th percentile of Branches Per 100k People: {branches_q25:.2f}")

```

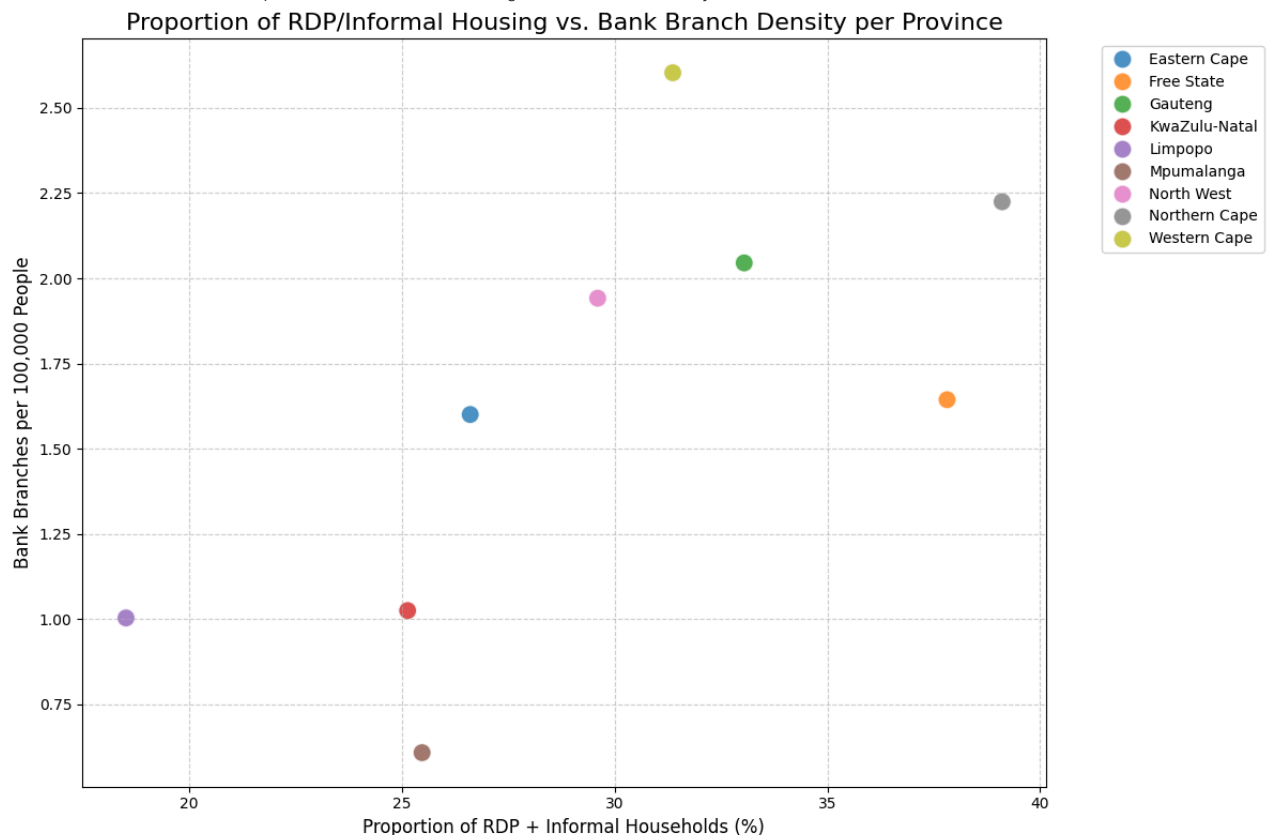
Proportion of RDP and Informal Housing per Province:

	Province_Name	Total_Weighted_Households	Weighted_RDP_Households	Weighted_Informal_Households	Prop_RDP	Prop_Informal	Prop_RDP_Informal
0	Eastern Cape	2.079473e+06	4.555428e+05	97792.272982	21.906646	4.702743	26.609388
1	Free State	8.957163e+05	2.745632e+05	64137.142886	30.652921	7.160430	37.813352
2	Gauteng	5.738113e+06	1.266923e+06	629298.689091	22.079090	10.966998	33.046087
3	KwaZulu-Natal	3.099370e+06	6.201932e+05	158954.962957	20.010298	5.128622	25.138920
4	Limpopo	1.918092e+06	2.759364e+05	79344.596317	14.385981	4.136641	18.522622

Merged Data (RDP/Informal Housing Proportion vs. Bank Branch Density per Province):

	Province_Name	Total_Weighted_Households	Weighted_RDP_Households	Weighted_Informal_Households	Prop_RDP	Prop_Informal	Prop_RDP_Informal	Total_Bank_Bran
0	Eastern Cape	2.079473e+06	4.555428e+05	97792.272982	21.906646	4.702743	26.609388	
1	Free State	8.957163e+05	2.745632e+05	64137.142886	30.652921	7.160430	37.813352	
2	Gauteng	5.738113e+06	1.266923e+06	629298.689091	22.079090	10.966998	33.046087	
3	KwaZulu-Natal	3.099370e+06	6.201932e+05	158954.962957	20.010298	5.128622	25.138920	
4	Limpopo	1.918092e+06	2.759364e+05	79344.596317	14.385981	4.136641	18.522622	

Pearson correlation between Proportion of RDP/Informal Housing and Bank Branch Density: 0.6736



--- Identified Service Gap Zones (High RDP/Informal, Low Branch Density) ---

Province_Name	Prop_RDP_Informal	Branches_Per_100k_People
---------------	-------------------	--------------------------

75th percentile of Proportion of RDP + Informal Households: 33.05%

25th percentile of Branches Per 100k People: 1.03

Hypothesis 5 Conclusion: based on the defined thresholds (top 25% for combined RDP/Informal housing and bottom 25% for bank branch density), no provinces were identified as clear 'Service Gap Zones'. This means that even in provinces with a high proportion of these housing types, the bank branch density was not among the lowest.

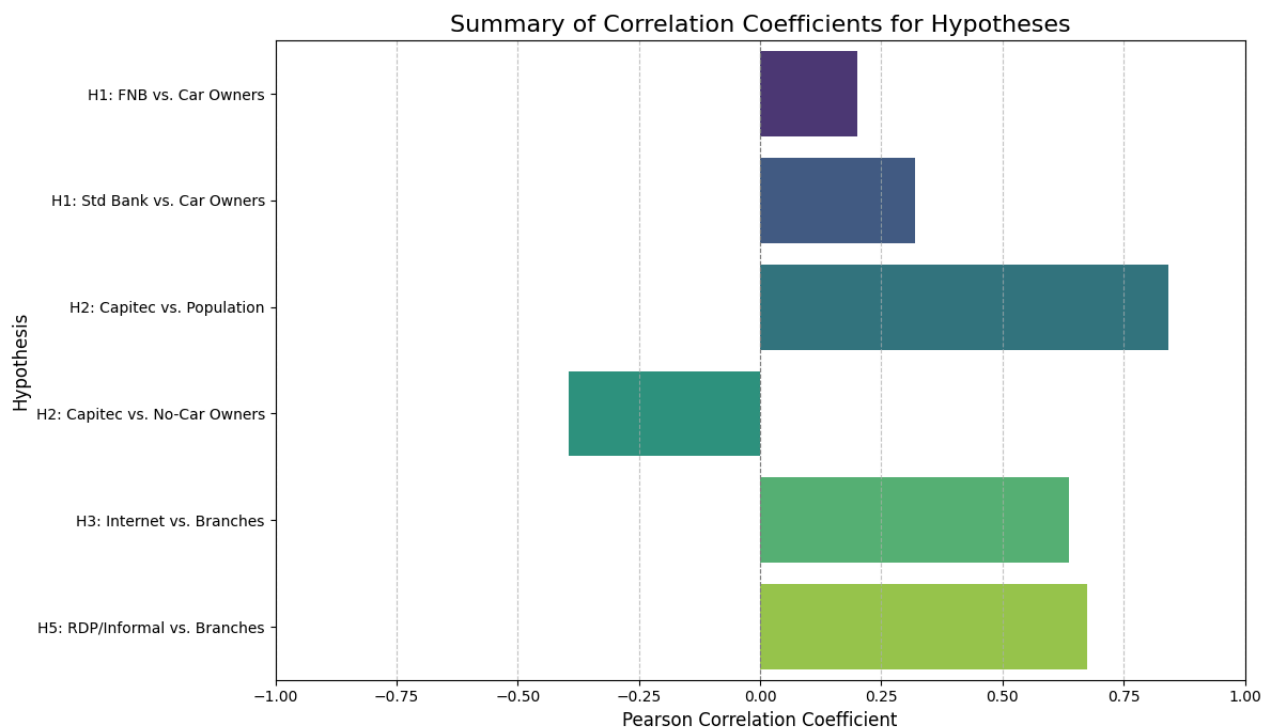
This finding might imply that banking services are still present in provinces with significant informal and RDP housing, possibly due to higher population density in some of these areas, or banks targeting these regions for growth and accessibility. It challenges the initial assumption that these areas would universally experience a 'spatial mismatch' in banking services at the provincial level.

```
In [60]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

correlation_data = {
    'Hypothesis': [
        'H1: FNB vs. Car Owners',
        'H1: Std Bank vs. Car Owners',
        'H2: Capitec vs. Population',
        'H2: Capitec vs. No-Car Owners',
        'H3: Internet vs. Branches',
        'H5: RDP/Informal vs. Branches'
    ],
    'Correlation_Coefficient': [
        correlation_fnb,
        correlation_standard_bank,
        correlation_capitec_population,
        correlation_capitec_no_car,
        correlation_internet_branches,
        correlation_rdp_informal_branches
    ]
}

corr_df = pd.DataFrame(correlation_data)

plt.figure(figsize=(12, 7))
sns.barplot(x='Correlation_Coefficient', y='Hypothesis', data=corr_df, palette='viridis', hue='Hypothesis', legend=False)
plt.axvline(0, color='gray', linestyle='--', linewidth=0.8) # Add a Line at 0 for reference
plt.title('Summary of Correlation Coefficients for Hypotheses', fontsize=16)
plt.xlabel('Pearson Correlation Coefficient', fontsize=12)
plt.ylabel('Hypothesis', fontsize=12)
plt.xlim(-1, 1) # Set x-axis Limits for correlation coefficients
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



Summary for Hypothesis 4: The 'Clustering' Hypothesis

Finding: Hypothesis 4, which states that traditional banks (FNB, ABSA, Nedbank) would show higher spatial clustering than Capitec, was **strongly supported**.

- **Traditional Banks:** Exhibited a higher degree of clustering, with a mean of approximately 1.67 and a median of 1 other branches within a 1km radius.
- **Capitec:** Showed a more diffuse distribution, with a mean of approximately 0.95 and a median of 0 other branches within a 1km radius.

This indicates that traditional banks tend to co-locate more frequently, likely in commercial hubs and shopping malls, while Capitec's branches are more spread out, consistent with a strategy targeting standalone locations in underserved or highly accessible community areas.

Which Provinces need more Bank Branches?

Based on this analysis, specifically looking at the bank branches per 100,000 people across provinces, some areas show a relatively lower density of physical bank branches. While our defined 'Service Gap Zones' and 'Banking Deserts' did not identify provinces that strictly met the criteria, we can observe which provinces have the fewest branches relative to their estimated population:

Mpumalanga has the lowest bank branch density, with approximately 0.61 branches per 100,000 people. Limpopo follows, with about 1.00 branches per 100,000 people. KwaZulu-Natal also has a relatively low density, with approximately 1.03 branches per 100,000 people. These provinces, particularly Mpumalanga, could be considered areas that relatively need more physical banking infrastructure based on population. It's important to note that this is a provincial-level analysis for 2022

```
In [68]: import folium
from folium.plugins import HeatMap
import geopandas as gpd

#target provinces
target_provinces = ['Mpumalanga', 'KwaZulu-Natal', 'Limpopo']

#province GeoDataFrame
filtered_provinces_gdf = sa_provinces_gdf[sa_provinces_gdf['adm1_name'].isin(target_provinces)].copy()

for col in filtered_provinces_gdf.select_dtypes(include=['datetime64', 'datetime64[ns]']).columns:
    filtered_provinces_gdf[col] = filtered_provinces_gdf[col].astype(str)

banks_in_target_provinces = gpd.sjoin(banks_gdf, filtered_provinces_gdf, how="inner", predicate='within')

center_lat = banks_in_target_provinces['lat'].mean() if not banks_in_target_provinces.empty else -28.7827
center_lon = banks_in_target_provinces['lon'].mean() if not banks_in_target_provinces.empty else 26.2934

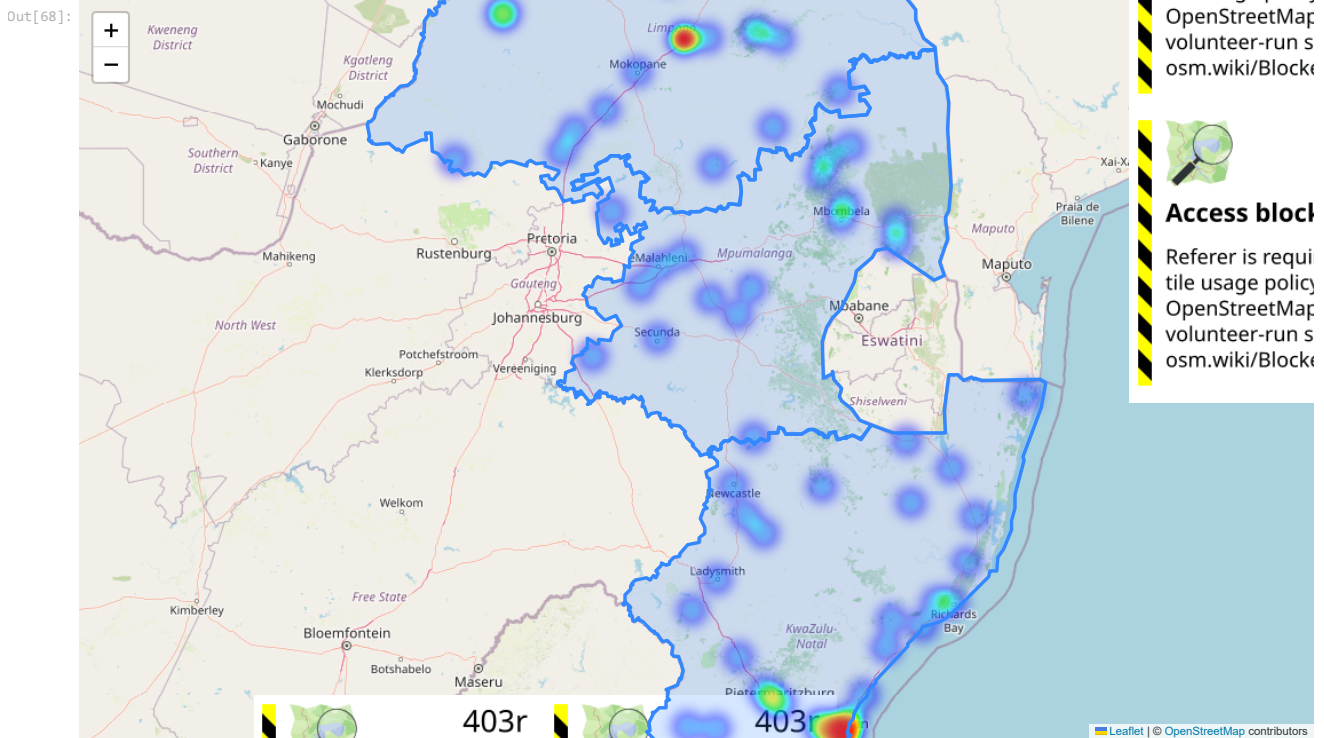
#map centered on the target provinces
m_filtered_heatmap = folium.Map(location=[center_lat, center_lon], zoom_start=7)

#adding the filtered province boundaries to the map
folium.GeoJson(filtered_provinces_gdf).add_to(m_filtered_heatmap)

#preparing data for HeatMap plugin (Latitude, Longitude, intensity)
heat_data_filtered = [[row['lat'], row['lon']] for index, row in banks_in_target_provinces.iterrows()]

if heat_data_filtered:
    HeatMap(heat_data_filtered, radius=15, blur=10, max_zoom=1).add_to(m_filtered_heatmap)

m_filtered_heatmap
```



```
In [63]: import geopandas as gpd

#GeoJSON file for municipality boundaries
mun_geojson_path = '/content/drive/MyDrive/Census2022/zaf_admin3.geojson'

try:
    sa_municipalities_gdf = gpd.read_file(mun_geojson_path)
    print("South African municipality boundaries loaded successfully from GeoJSON. Displaying head:")
    display(sa_municipalities_gdf.head())
    print(f"Number of municipalities found: {len(sa_municipalities_gdf)}")
except FileNotFoundError:
    print(f"Error: The GeoJSON file '{mun_geojson_path}' was not found. Please check the path.")
except Exception as e:
    print(f"An error occurred while loading the GeoJSON: {e}")
```

South African municipality boundaries loaded successfully from GeoJSON. Displaying head:

	adm3_name	adm3_name1	adm3_name2	adm3_name3	adm3_pcode	adm2_name	adm2_name1	adm2_name2	adm2_name3	adm2_pcode	...	area_sqkm	version
0	Abaqulusi	KZN263	None	None	ZA5263	Zululand	DC26	None	None	ZA526	...	4313.758286	v01
1	Alfred Duma	KZN238	None	None	ZA5238	Uthukela	DC23	None	None	ZA523	...	3764.472496	v01
2	Amahlathi	EC124	None	None	ZA2124	Amathole	DC12	None	None	ZA212	...	4505.477231	v01
3	Ba-Phalaborwa	LIM334	None	None	ZA9334	Mopani	DC33	None	None	ZA933	...	7489.282696	v01
4	Beaufort West	WC053	None	None	ZA1053	Central Karoo	DC5	None	None	ZA105	...	21916.602218	v01

5 rows × 32 columns

Number of municipalities found: 213

```
In [65]: import geopandas as gpd

if 'banks_gdf' not in locals() or not isinstance(banks_gdf, gpd.GeoDataFrame):
    banks_gdf = banks_df.dropna(subset=['lat', 'lon']).copy() # Assuming banks_df is available
    banks_gdf['geometry'] = gpd.points_from_xy(banks_gdf['lon'], banks_gdf['lat'], crs="EPSG:4326")
    banks_gdf = gpd.GeoDataFrame(banks_gdf, geometry='geometry', crs="EPSG:4326")

#List of target provinces
target_provinces = ['Mpumalanga', 'KwaZulu-Natal', 'Limpopo']

#filter bank branches to only include those in the target provinces
#first, need to spatially join banks with provinces to assign province names to banks
banks_with_province = gpd.sjoin(banks_gdf, sa_provinces_gdf[['adm1_name', 'geometry']], how="inner", predicate='within', lsuffix='left_prov', rsuffix='right_prov')
banks_in_target_provinces = banks_with_province[banks_with_province['adm1_name'].isin(target_provinces)].copy()

if sa_municipalities_gdf.crs != banks_in_target_provinces.crs:
    sa_municipalities_gdf = sa_municipalities_gdf.to_crs(banks_in_target_provinces.crs)

banks_per_municipality = gpd.sjoin(banks_in_target_provinces, sa_municipalities_gdf[['adm3_name', 'geometry']], how="inner", predicate='within', lsuffix='left_prov', rsuffix='right_prov')

#the number of banks in each municipality
municipality_bank_counts = banks_per_municipality.groupby('adm3_name').size().sort_values(ascending=True).reset_index(name='Num_Bank_Branches')

print("Number of bank branches per municipality in Mpumalanga, KwaZulu-Natal, and Limpopo (sorted by fewest branches):")
display(municipality_bank_counts.head(10))

print("Municipalities with 0 bank branches:")
#ID municipalities in target provinces with zero banks
#a list of all municipalities in the target provinces
mun_in_target_provinces = sa_municipalities_gdf[sa_municipalities_gdf['adm1_name'].isin(target_provinces)]

municipalities_with_no_banks = mun_in_target_provinces.merge(
    municipality_bank_counts,
    left_on='adm3_name',
    right_on='adm3_name',
    how='left'
)

municipalities_with_no_banks['Num_Bank_Branches'] = municipalities_with_no_banks['Num_Bank_Branches'].fillna(0).astype(int)
zero_bank_muns = municipalities_with_no_banks[municipalities_with_no_banks['Num_Bank_Branches'] == 0] # Corrected this line

display(zero_bank_muns[['adm3_name', 'adm1_name', 'Num_Bank_Branches']])
```

Number of bank branches per municipality in Mpumalanga, KwaZulu-Natal, and Limpopo (sorted by fewest branches):

	adm3_name	Num_Bank_Branches
0	Alfred Duma	1
1	Big Five Hlabisa	1
2	Dannhauser	1
3	Dr Pixley Ka Isaka Seme	1
4	Endumeni	1
5	Molemole	1
6	Msukaligwa	1
7	Maruleng	1
8	Mpofana	1
9	Umhlabuyalingana	1

Municipalities with 0 bank branches:

	adm3_name	adm1_name	Num_Bank_Branches
2	Ba-Phalaborwa	Limpopo	0
8	Collins Chabane	Limpopo	0
11	Dr JS Moroka	Mpumalanga	0
14	eDumbe	KwaZulu-Natal	0
15	Elias Motsoaledi	Limpopo	0
16	Emadlangeni	KwaZulu-Natal	0
17	Emakhazeni	Mpumalanga	0
20	Ephraim Mogale	Limpopo	0
23	Greater Giyani	Limpopo	0
24	Greater Kokstad	KwaZulu-Natal	0
25	Greater Letaba	Limpopo	0
28	Impendle	KwaZulu-Natal	0
29	Inkosi Langalibalele	KwaZulu-Natal	0
32	Lekwa	Mpumalanga	0
33	Lepele-Nkumpi	Limpopo	0
38	Maphumulo	KwaZulu-Natal	0
41	Mfolozi	KwaZulu-Natal	0
42	Mkhambathini	KwaZulu-Natal	0
43	Mkhondo	Mpumalanga	0
48	Msinga	KwaZulu-Natal	0
50	Mthonjaneni	KwaZulu-Natal	0
52	Musina	Limpopo	0
53	Ndwedwe	KwaZulu-Natal	0
55	Nkandla	KwaZulu-Natal	0
58	Nqutu	KwaZulu-Natal	0
62	Richmond	KwaZulu-Natal	0
70	Ulundi	KwaZulu-Natal	0
71	Umdoni	KwaZulu-Natal	0
76	uMshwathi	KwaZulu-Natal	0
77	uMuziwabantu	KwaZulu-Natal	0
78	Umvoti	KwaZulu-Natal	0
79	Umkhulu	KwaZulu-Natal	0
80	Umzumbe	KwaZulu-Natal	0
82	Victor Khanye	Mpumalanga	0

The tables above show the municipalities in Mpumalanga, KwaZulu-Natal, and Limpopo with the fewest bank branches. Municipalities with 0 branches are explicitly listed. These areas could be considered for potential new bank branch placement, especially when considering population density and socio-economic indicators from the municipal level.

```
In [69]: import folium
import geopandas as gpd

#filteringg for municipalities in the target provinces if not already done
target_provinces_gdf = sa_municipalities_gdf[sa_municipalities_gdf['adm1_name'].isin(target_provinces)].copy()

#ensuring the 'Num_Bank_Branches' column is correctly merged for all municipalities in target provinces
#Re-merging to ensure all target municipalities are present, even those with no banks initially
municipalities_for_map = target_provinces_gdf.merge(
    municipality_bank_counts,
    left_on='adm3_name',
    right_on='adm3_name',
    how='left'
)
municipalities_for_map['Num_Bank_Branches'] = municipalities_for_map['Num_Bank_Branches'].fillna(0).astype(int)

for col in municipalities_for_map.select_dtypes(include=['datetime64', 'datetime64[ns]']).columns:
    municipalities_for_map[col] = municipalities_for_map[col].astype(str)

center_lat_mun = municipalities_for_map.geometry.centroid.y.mean()
center_lon_mun = municipalities_for_map.geometry.centroid.x.mean()

m_zero_banks = folium.Map(location=[center_lat_mun, center_lon_mun], zoom_start=7)

def style_function(feature):
    num_branches = feature['properties'].get('Num_Bank_Branches', 0)
    if num_branches == 0:
        return {
            'fillColor': '#e31a1c', # Red for zero branches
            'color': 'black',
            'weight': 1,
            'fillOpacity': 0.7
        }
    else:
        return {
            'fillColor': '#336699', # Blue for municipalities with branches
```

```

        'color': 'black',
        'weight': 1,
        'fillOpacity': 0.5
    }

folium.GeoJson(
    municipalities_for_map,
    style_function=style_function,
    tooltip=folium.GeoJsonTooltip(fields=['adm3_name', 'Num_Bank_Branches'], aliases=['Municipality', 'Bank Branches'], localize=True)
).add_to(m_zero_banks)

legend_html = '''
<div style="position: fixed;
    bottom: 50px; left: 50px; width: 180px; height: 80px;
    border: 2px solid grey; z-index: 9999; font-size: 14px;
    background-color: white; opacity: 0.9;
">
    &nbsp; <b>Municipalities</b> <br>
    &nbsp; <i style="background: #e31a1c; display: inline-block; width: 10px; height: 10px;"></i> &nbsp; 0 Bank Branches <br>
    &nbsp; <i style="background: #336699; display: inline-block; width: 10px; height: 10px;"></i> &nbsp; Has Bank Branches <br>
</div>
'''

m_zero_banks.get_root().html.add_child(folium.Element(legend_html))

m_zero_banks

```

/tmp/ipykernel_45077/367922060.py:22: UserWarning: Geometry is in a geographic CRS. Results from 'centroid' are likely incorrect. Use 'GeoSeries.to_crs()' to re-project geometries to a projected CRS before this operation.

center_lat_mun = municipalities_for_map.geometry.centroid.y.mean()
/tmp/ipykernel_45077/367922060.py:23: UserWarning: Geometry is in a geographic CRS. Results from 'centroid' are likely incorrect. Use 'GeoSeries.to_crs()' to re-project geometries to a projected CRS before this operation.

center_lon_mun = municipalities_for_map.geometry.centroid.x.mean()

Out[69]:

